

CSS: Transitions

- Permite modelar a mudança de propriedades entre os elementos, tornando-os animados.



Alterando o tamanho da `<div>`



Alterando a posição da `<div>`

CSS: Transitions

- Sintaxe:
 - transition: **nome_evento | duração | efeito**
 - transition: background-color 1s ease-in
- Efeitos: ease-in, ease-out e ease-in-out
 - ease-in: começa devagar e acelera até o fim
 - ease-out: o oposto, começa rápido e desacelera até o fim
 - ease-in-out: começa devagar, acelera no meio e reduz no fim

- Exemplo simples

```
<a class="target">Passe o mouse aqui !!!</a>
```

```
.target {  
  font-size: 14px;  
  transition: font-size 4s 1s;  
}
```

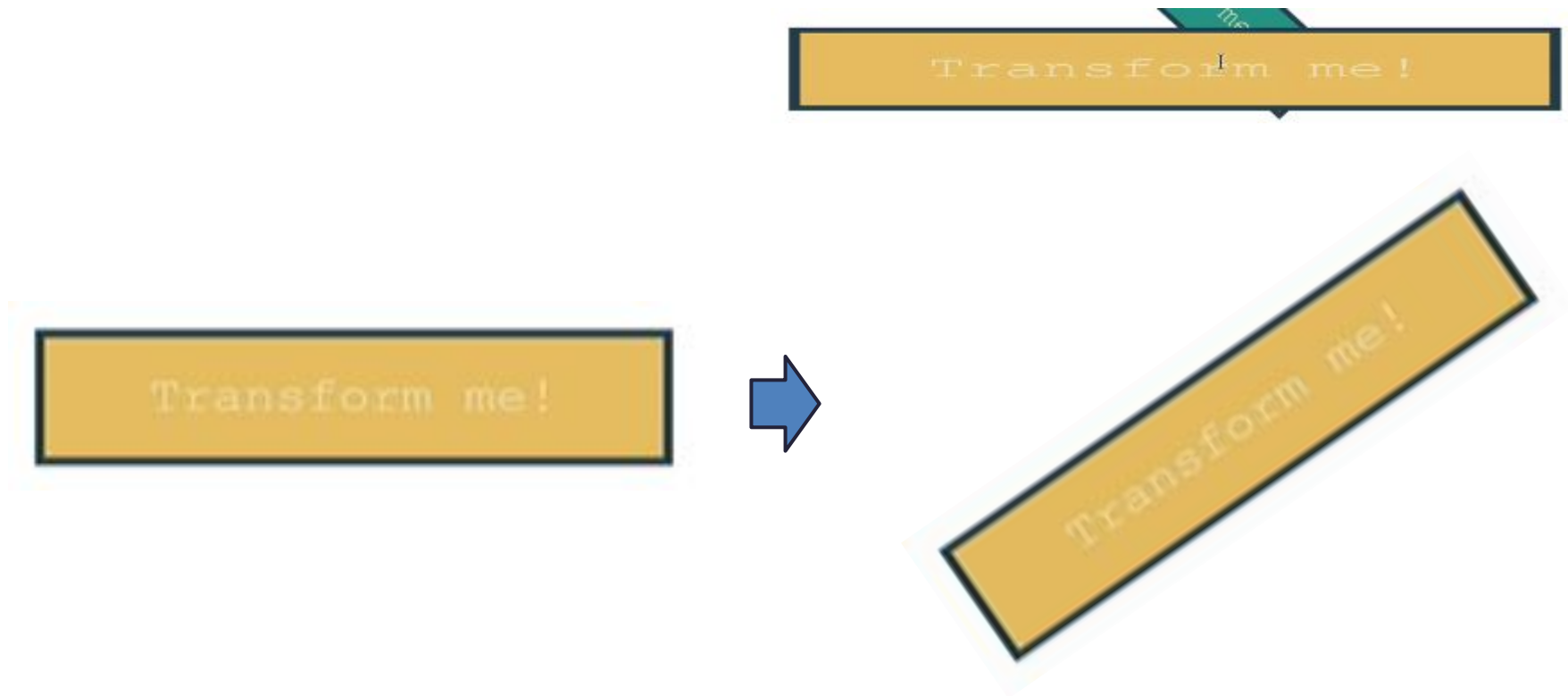
```
.target:hover {  
  font-size: 36px;  
}
```

CSS: Transitions

[illegible]

CSS: Transformations

- O comando *transform* permite você girar, dimensionar, inclinar ou posicionar um elemento.



CSS: Transformations

- Propriedade
 - rotate()

Rotaciona o elemento

```
div {  
  border: solid red;  
  transform: translate(30px, 20px) rotate(20deg);  
  width: 140px;  
  height: 60px;  
}
```

Move o elemento

<div>Transformed element</div>



CSS: Transformations

- Propriedade

- scale()

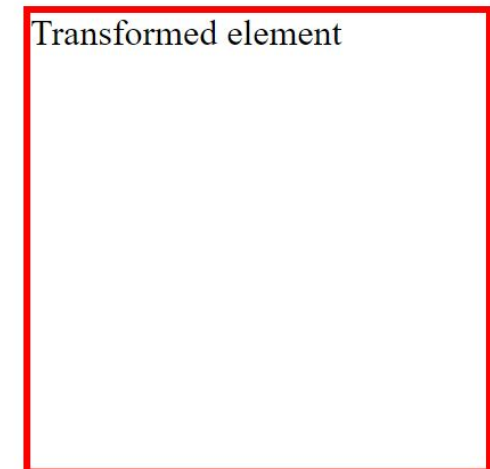
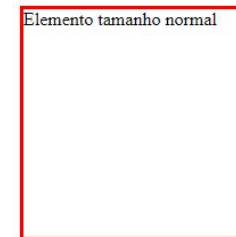
- Permite aumentar ou diminuir o elemento

```
div {  
  border: solid red;  
  width: 200px;  
  height: 200px;  
}
```

```
#transformed{  
  border: solid red;  
  transform: translate(200px, 200px) scale(2);  
  width: 200px;  
  height: 200px;  
}
```

```
<div>Elemento tamanho normal</div>
```

```
<div id="transformed">Transformed element</div>
```



CSS: Transformations

- Propriedade

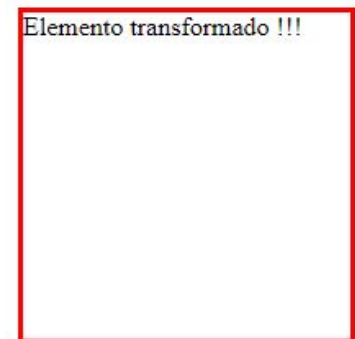
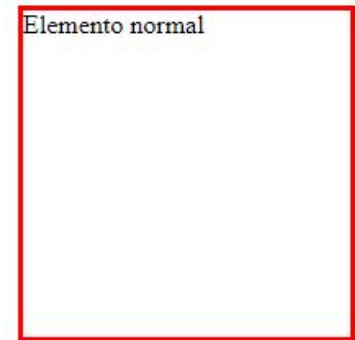
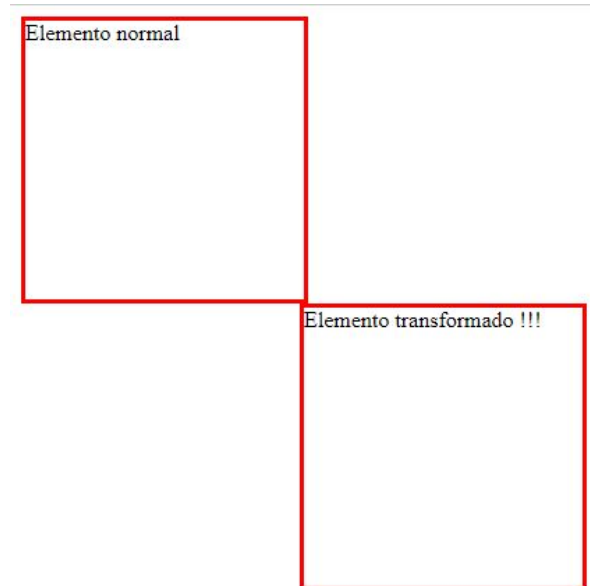
- `translate(x,y)`, `translateX(x)`, `translateY(y)`

- Permite movimentar um elemento

```
div {  
  border: solid red;  
  width: 200px;  
  height: 200px;  
}  
  
#transformed{  
  border: solid red;  
  transform: translate(200px, 200px);  
  width: 200px;  
  height: 200px;  
}
```

```
<div>Elemento normal</div>
```

```
<div id="transformed">Elemento transformado !!!</div>
```



CSS: Transformations

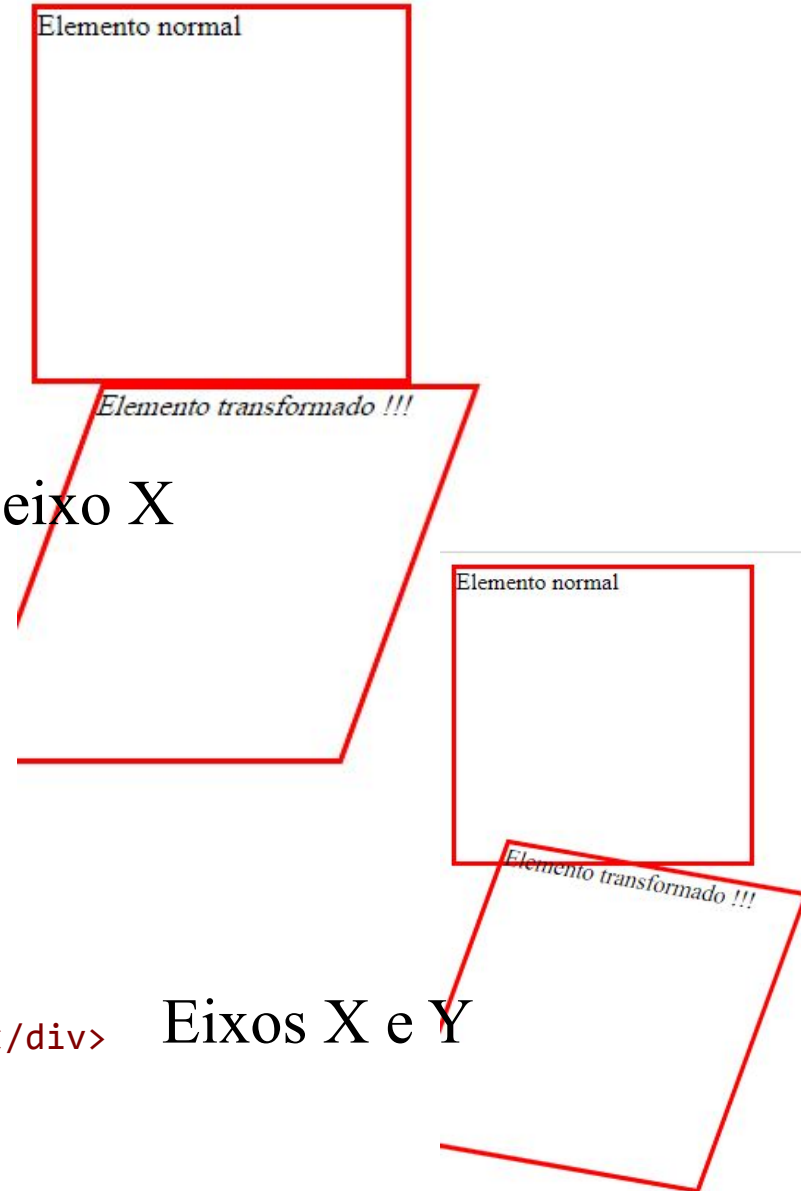
- Propriedade
 - skew(grausX, grausY)
 - Permite inclinar o elemento

```
div {  
  border: solid red;  
  width: 200px;  
  height: 200px;  
}  
  
#transformed{  
  border: solid red;  
  transform: skew(-20deg);  
  width: 200px;  
  height: 200px;  
}
```

<div>Elemento normal</div>

<div id="transformed">Elemento transformado !!!</div>

Somente eixo X



Eixos X e Y

CSS: Transformations

- Propriedade

- `cursor: pointer;`

- Obs.: Também é possível criar cursores personalizados!

<https://developer.mozilla.org/en-US/docs/Web/CSS/cursor>

	default
	crosshair
	hand
	pointer
	Cross browser
	move
	text
	wait
	help

	n-resize
	ne-resize
	e-resize
	se-resize
	s-resize
	sw-resize
	w-resize
	nw-resize
	progress

	not-allowed
	no-drop
	vertical-text
	all-scroll
	col-resize
	row-resize

CSS: Transformations

- Propriedade
 - `box-shadow: COR TAM1 TAM2;`
 - Também permite a inclusão de mais 2 tamanhos, TAM3 e TAM4, representando efeito borrado e disperso.

<https://developer.mozilla.org/en-US/docs/Web/CSS/box-shadow>

```
box-shadow: red 10px 10px;
```

This is a box with a box-shadow around it.

CSS: Transformations

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Exemplos de CSS Transformations</title>
  <style>
    body {
      font-family: 'Segoe UI', sans-serif;
      background-color: #eef2f3;
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
      gap: 30px;
      padding: 50px;
    }

    .box {
      background: white;
      padding: 40px;
      text-align: center;
      border-radius: 12px;
      border: 1px solid #ddd;
      box-shadow: 0 4px 6px rgba(0,0,0,0.05);
      transition: all 0.4s ease; /* Para suavizar a transformação */
      cursor: pointer;
    }

    .box span {
      display: block;
      font-size: 40px;
      margin-bottom: 10px;
    }

    /* 1. SCALE: Zoom suave */
    .zoom:hover {
      transform: scale(1.15);
      border-color: #54986b;
    }

    /* 2. ROTATE: Giro completo */
    .girar:hover {
      transform: rotate(360deg);
      background-color: #f1c40f;
    }

    /* 3. TRANSLATE: Movimento lateral */
    .mover:hover {
      transform: translateX(30px);
      background-color: #74b3c;
      color: white;
    }

    /* 4. SKEW: Inclinação/Distorção */
    .inclinar:hover {
      transform: skewX(-15deg);
      background-color: #9b59b6;
      color: white;
    }

    /* 5. COMBINADO: Escala + Rotação */
    .combo:hover {
      transform: scale(0.8) rotate(-15deg);
      background-color: #2ecc71;
      color: white;
    }
  </style>
</head>
<body>

  <div class="box zoom">
    <span></span>
    <strong>Scale</strong>
    <p>Aumenta o tamanho</p>
  </div>

  <div class="box girar">
    <span></span>
    <strong>Rotate</strong>
    <p>Gira o elemento</p>
  </div>

  <div class="box mover">
    <span></span>
    <strong>Translate</strong>
    <p>Move para o lado</p>
  </div>

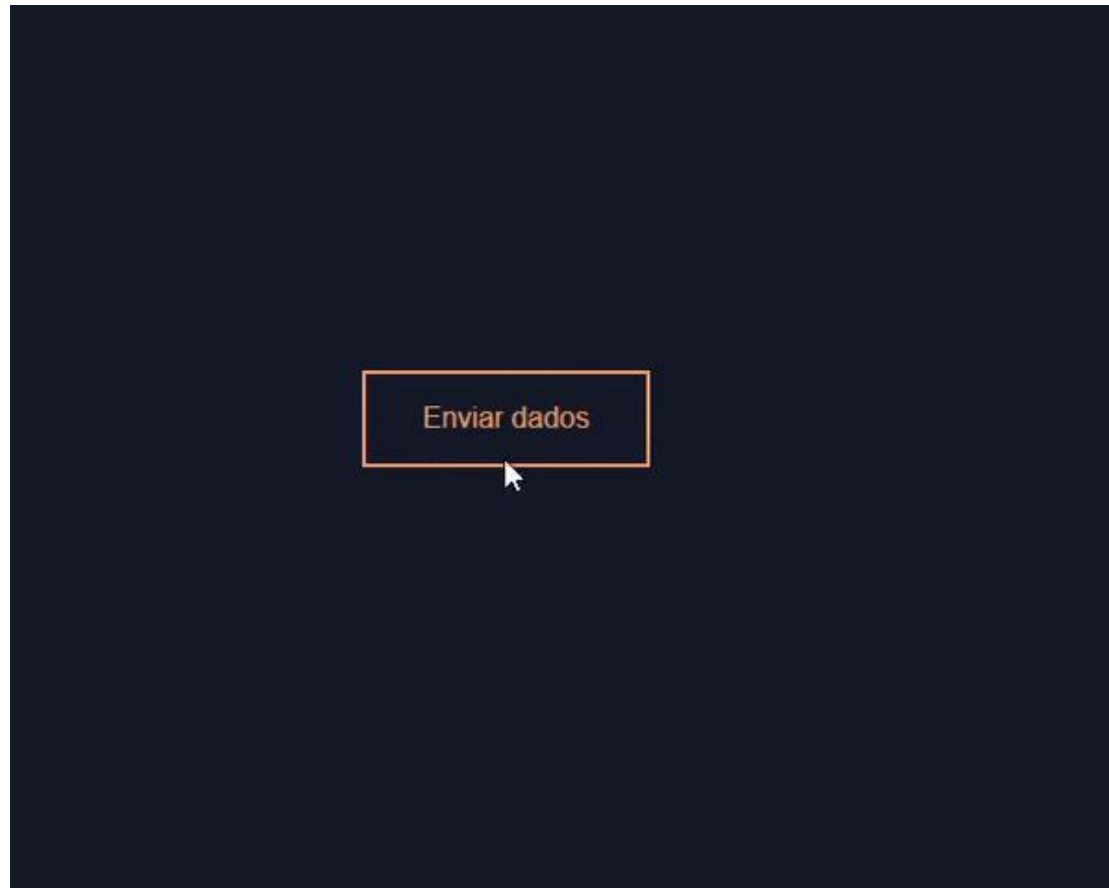
  <div class="box inclinar">
    <span></span>
    <strong>Skew</strong>
    <p>Distorce os ângulos</p>
  </div>

  <div class="box combo">
    <span></span>
    <strong>Combo</strong>
    <p>Múltiplas transformações</p>
  </div>

</body>
</html>
```

Atividade

- Quais componentes CSS Podemos utilizar?



CSS: Botão Estilizado - Etapa 1

- HTML
 - `<button>Enviar dados</button>`
- CSS

```
body {  
    font-family: 'Roboto',  
    sans-serif;  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    height: 100vh;  
    background-color:  
    #151b29;  
}
```

CSS: Botão Estilizado - Etapa 2

```
button {  
  background: none;  
  color: #ffa260;  
  border: 2px solid;  
  padding: 1em 2em;  
  font-size: 1em;  
  transition: color 0.25s, border-color 0.25s,  
box-shadow 0.25s, transform 0.25s;  
}
```

```
button:hover {  
  border-color: #f1ff5c;  
  color: white;  
  box-shadow: 0 0.5em 0.5em -0.4em #f1ff5c;  
  transform: translateY(-0.9em);  
  cursor: pointer;  
}
```

CSS: background-image

- É possível definir mais de uma imagem no background

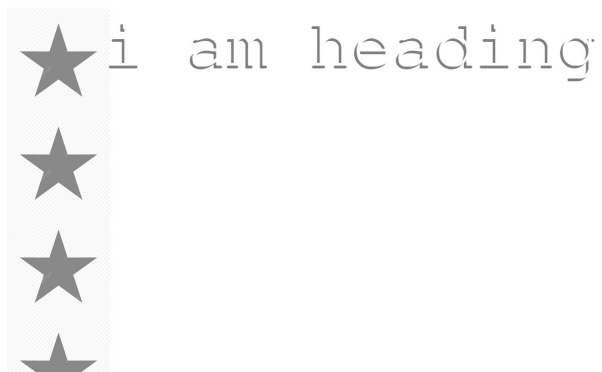
```
background-image: url("circle.png"),  
url("star.png");  
background-position: top left, bottom right;  
background-repeat: no-repeat, no-repeat;
```

 i am heading



CSS: background-image

```
background: url(star.png)  
repeat-x;
```



```
background: url(star.png)  
repeat-y;
```

CSS: background-image

Preencha todo o espaço (ou 70% dele)

```
background: center/cover url("star.png");  
background: center/70% url("star.png");
```



CSS: layout

- Dividir a página em áreas usando o elemento div ou section do HTML.
- Cada área deve ter um nome bem representativo e não vinculado a informações sobre o estilo ou posicionamento a ser aplicado.
 - O estilo e o posicionamento podem ser modificados e o nome dado pode perder o sentido original.
 - Exemplos de “bons” nomes:
 - geral, container, menu, conteudo, destaque, rodape, footer, blogs, topo, centro, center, etc.

CSS: layout

- Estruturar as áreas que esperamos no nosso layout.

```
<div id="container">  
  <div id="header"></div>  
  <div id="left"></div>  
  <div id="content"></div>  
  <div id="right"></div>  
  <div id="footer"></div>  
</div>
```

CSS: layout

- Número de colunas
- Largura da página
 - Fixa
 - A largura é fixa e não se expande ou se encolhe de acordo com a janela do navegador.
 - Melhor controle do design.
 - Líquida
 - O conteúdo da página se expande para se ajustar à página de acordo com as dimensões da janela do navegador.
 - Elástica
 - A largura é fixa, mas as margens se expandem ou se encolhem de acordo com a janela do navegador.
 - Normalmente posiciona o conteúdo no centro da página.

CSS: layout de largura fixa

```
#container {  
  border: 1px solid black;  
  width: 500px;  
  background-color: rgb(255,108, 180)  
}
```

Mesmo que você redimensione o tamanho da janela, será de 500px o elemento!

CSS: layout de largura líquida

```
#container {  
  border: 1px solid black;  
  width: 80%;  
  min-width: 300px;  
}
```

Vai ocupar 80% do espaço reservado ao elemento.

CSS: layout de largura elástica

```
/* largura elastica */  
  
* {  
    font-size: 15px;  
}  
  
#container {  
    border: 1px solid black;  
    width: 30em;  
    /* de acordo com o tamanho da fonte */  
}
```


CSS: layout (centralização)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Exemplo Prático de Margin CSS</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
    }

    /* ----- */
    /* 1. EXEMPLO DE CENTRALIZAÇÃO HORIZONTAL */
    /* ----- */
    .caixa-centralizada {
      /* 1. Define uma largura para a caixa */
      width: 70%;
      background-color: #3498db;
      color: white;
      padding: 15px;
      border: 3px solid #2980b9;
      text-align: center;

      /* 2. O segredo: 'auto' nas margens laterais centraliza o bloco */
      margin: 50px auto;
      /* 50px para Topo e Inferior, e 'auto' para Esquerda e Direita */
    }

    /* ----- */
    /* 2. EXEMPLO DE MARGEM SHORTHAND */
    /* ----- */
    .caixa-margens {
      width: 400px;
      background-color: #2ecc71;
      color: black;
      padding: 10px;
      border: 2px solid #27ae60;

      /* Quatro valores: Topo(10px) Direita(50px) Inferior(30px) Esquerda(5px) */
      margin: 10px 50px 30px 5px;
    }
  </style>
</head>
<body>

  <div class="caixa-centralizada">
    Este elemento está centralizado horizontalmente.
    (Usa: margin: 50px auto;)
  </div>

  <div class="caixa-margens">
    Este elemento demonstra o "shorthand" de margens, criando um deslocamento irregular:
    <br>
    Topo: 10px, Direita: 50px, Inferior: 30px, Esquerda: 5px
  </div>
```

CSS: layout

- min-width
 - Define a largura mínima do elemento.
- min-height
 - Define a altura mínima do elemento.

```
div {  
    max-width: 500px;  
    height: 100px;  
    background-color: powderblue;  
}
```

<h2>Set the max-width of an element</h2>

<p>This div element has a height of 100px and a max-width of 500px:</p>

<div></div>

<p>Resize the browser window to see the effect.</p>

Até a próxima!



Encerramento CSS - Aula 4

CSS: Google font-family

- A maioria das fontes mais comuns estão presentes em praticamente todos os navegadores, porém, podemos definir uma sequência de fontes que devem ser usadas caso a anterior não seja encontrada.
- Além disso, é possível usar fontes customizadas, como as oferecidas pelo Google.

CSS: Google font-family

- Aproximadamente há 900 fontes disponíveis gratuitamente.
- <https://fonts.google.com/>
- Após selecionar as fontes que deseja no site, é gerado um código CSS com a importação das fontes que deve ser adicionado na sua aplicação.

EXEMPLO DE IMPORTAÇÃO

```
<link rel="preconnect" href="https://fonts.googleapis.com">  
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>  
<link href="https://fonts.googleapis.com/css2?family=Workbench&display=swap"  
rel="stylesheet">
```

CUSTOMIZAÇÃO:

```
.workbench-<uniquifier> {  
font-family: "Workbench", sans-serif;  
font-optical-sizing: auto;  
font-weight: 400;  
font-style: normal;  
font-variation-settings: "BLED" 0, "SCAN" 0;  
}
```

CSS: Google font-family

Esta é uma tag de paragrafo usando a fonte importada do Google chamada de Workbench!

```
p{  
    font-family: Workbench;  
}
```

```
<body>
```

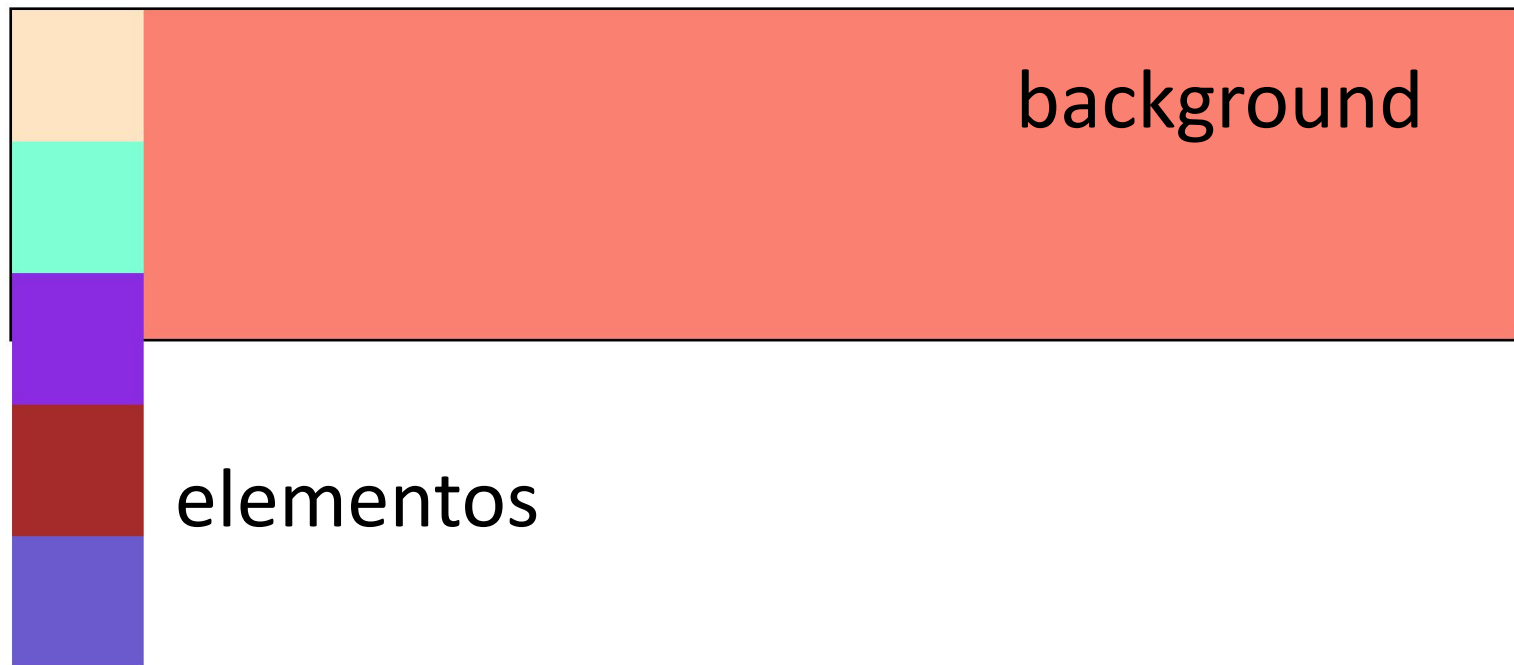
```
    <p>Esta é uma tag de paragrafo usando a  
    fonte importada do Google chamada de  
    Workbench!</p>
```

```
</body>
```

CSS responsivo e Flexbox

- Flexbox fornece um ambiente adaptável a diferentes cenários, tornando-o responsivo (adapta-se a diferentes ambientes – tamanhos de tela, organização de componentes, etc)
- Flexbox é um sistema de layout em CSS projetado para facilitar o posicionamento e a organização de elementos em uma página da web de forma mais eficiente e previsível do que os modelos de layout tradicionais.
- Com o Flexbox, você pode criar layouts responsivos e flexíveis, onde os elementos dentro de um contêiner podem ser distribuídos, alinhados e redimensionados automaticamente de acordo com as propriedades definidas.
- Flexbox permite a distribuição do espaço da tela entre os elementos de diferentes tamanhos de forma dinâmica, daí o termo flex.

- Sem uso do Flexbox (elementos são alinhados em bloco)



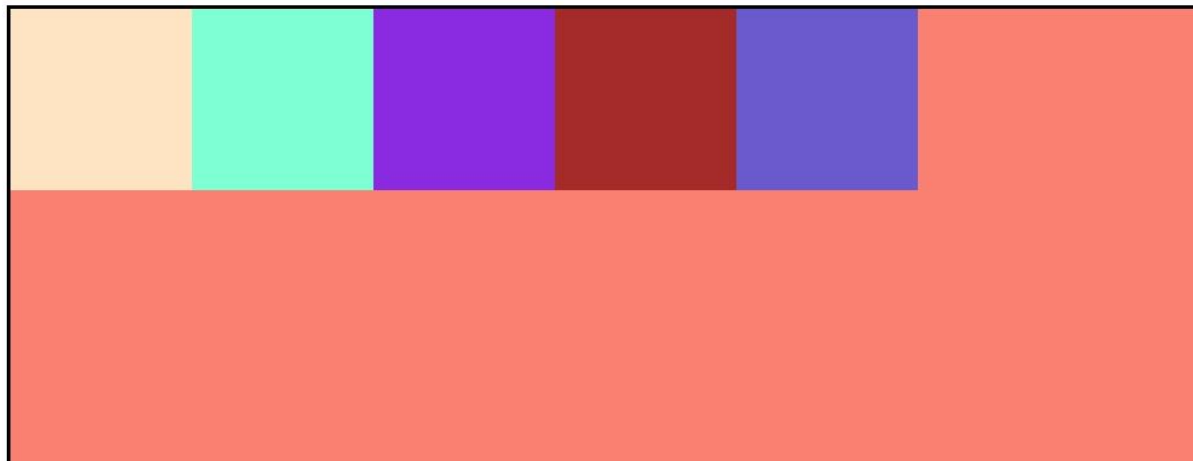
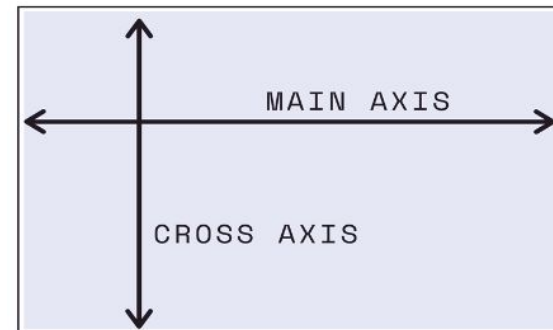
Flexbox

Ao usar o “display: flex”

```
<section id="container">
  <div style="background-color: bisque;"></div>
  <div style="background-color: aquamarine;"></div>
  <div style="background-color: blueviolet;"></div>
  <div style="background-color: brown;"></div>
  <div style="background-color: slateblue;"></div>
</section>
```

```
#container{
  background-color: salmon;
  width: 90%;
  height: 500px;
  border: 5px solid black;
  /*display: flex;*/
  margin: 0 auto;
}
```

```
#container div{
  width: 200px;
  height: 200px;
}
```

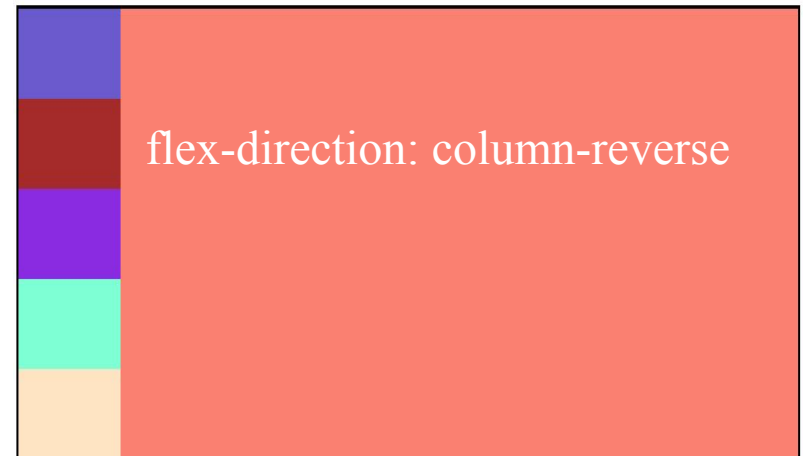
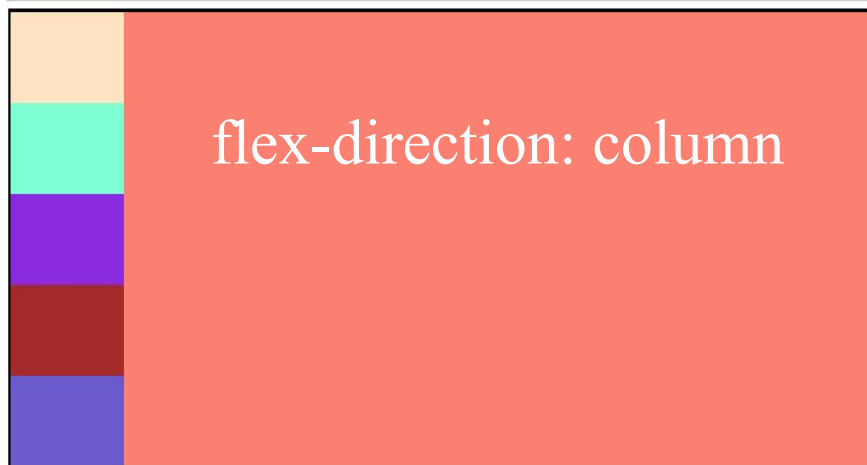
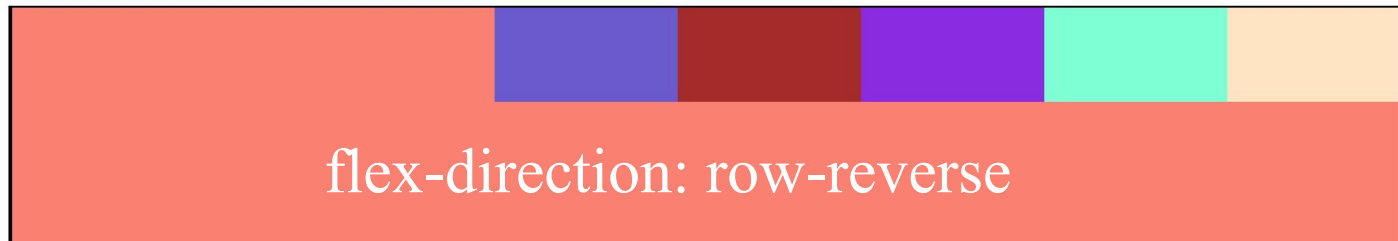
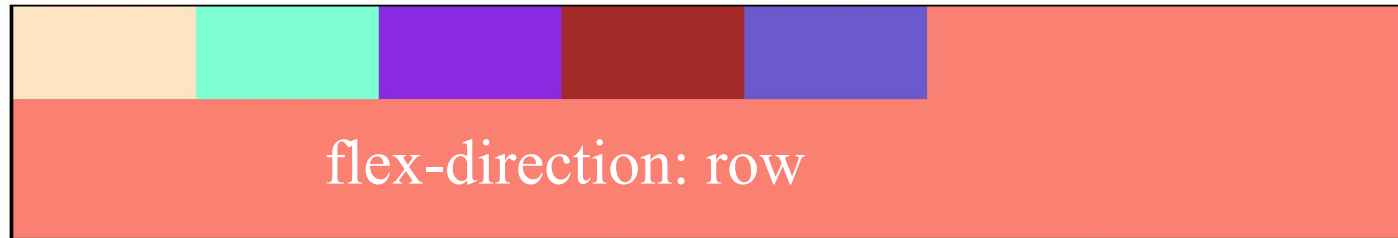


Flexbox: flex-direction

- Ao ativar o Flexbox com o “display: flex” é possível organizar os elementos de forma dinâmica.
- Da esquerda para a direita por padrão.
- Com o atributo “flex-direction” é possível alterar a ordem da organização dos elementos
 - row (default)
 - row-reverse
 - column
 - column-reverse

Flexbox: flex-direction

Se o espaço do nosso container for menor que o tamanho necessário para alocar todos os itens, eles serão redimensionados. Se o tamanho do container for maior que o necessário, teremos espaço livre.



Flexbox: justify-content

- Valores
 - flex-start
 - flex-end
 - center
 - space-between
 - space-around



```
justify-content: flex-start;
```



```
justify-content: flex-end;
```



```
justify-content: center;
```



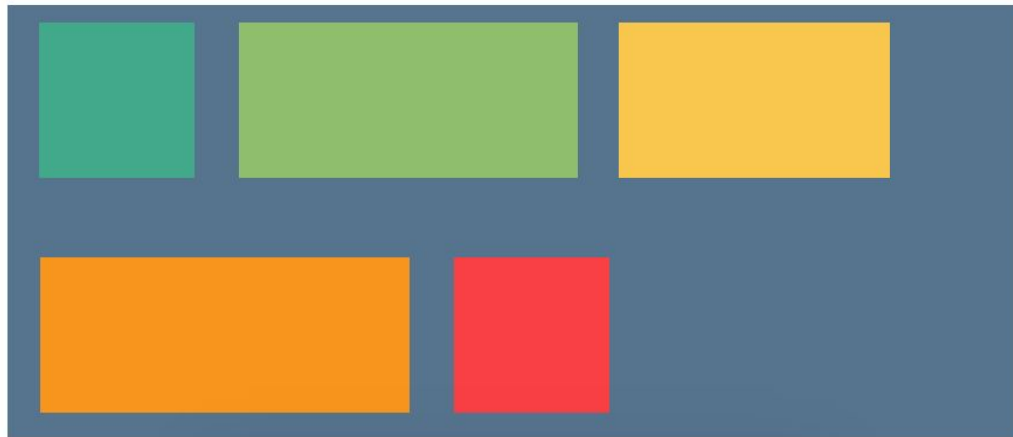
```
justify-content: space-between;
```



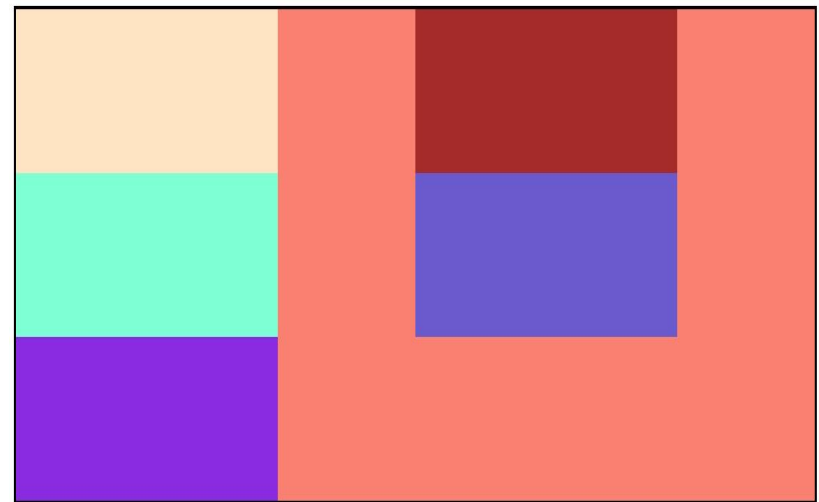
```
justify-content: space-around;
```

Flex-box: flex-wrap

Permite que todos os itens usem seu tamanho original, sem redimensionamento. Eles serão reposicionados para baixo (`flex-direction: row`) ou para os lados (`flex-direction: column`).



```
flex-wrap: wrap;
```



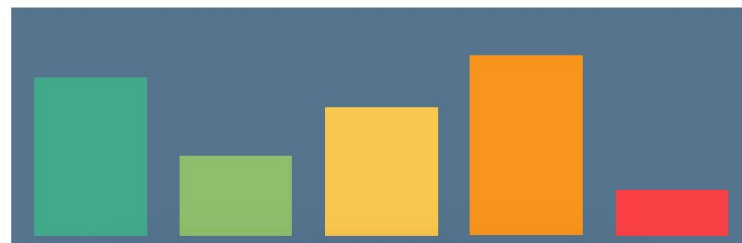
Flex-box: align-items

- Valores
 - flex-start
 - flex-end
 - center
 - stretch

Flex-box: align-items



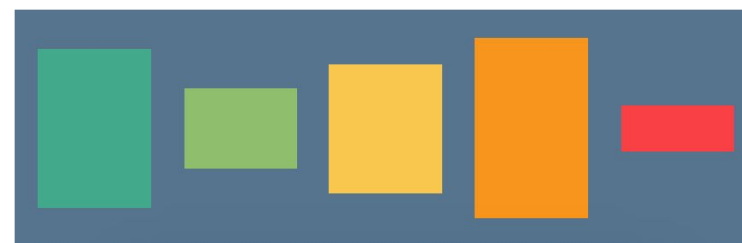
```
align-items: flex-start;
```



```
align-items: flex-end;
```

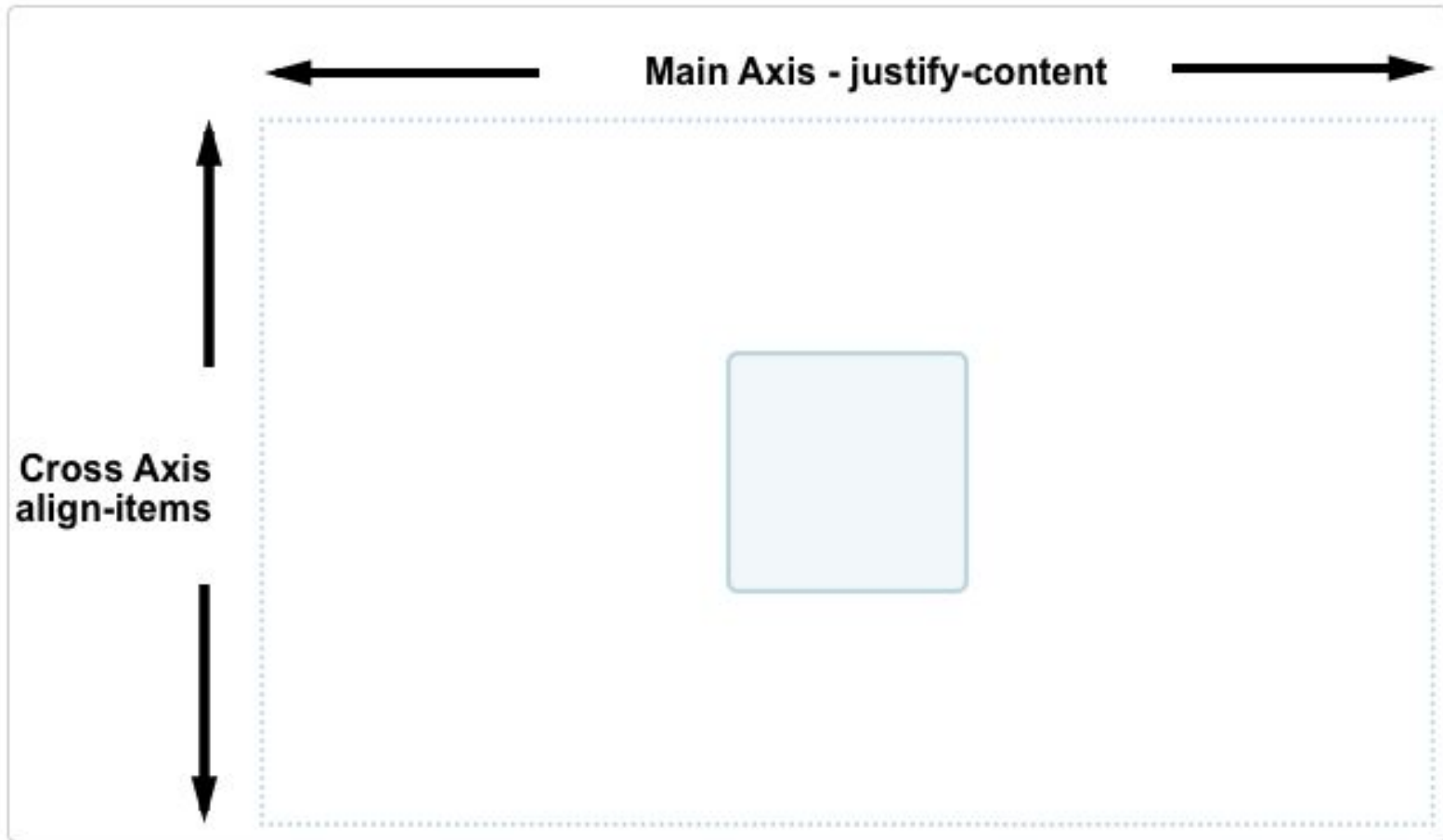


```
align-items: stretch;
```



```
align-items: center;
```


Flex-box: justify-content vs align-items



https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout/Aligning_items_in_a_flex_container

Flex-box: align-items

Para usar o “stretch” é preciso que o width ou o height estejam vazios, se ele estiver especificado o elemento não será esticado.

`align-items: stretch;`

- Exige `height` vazio e `flex-direction: row;`
- Exige `width` vazio e `flex-direction: column;`



```
align-items: stretch;
```

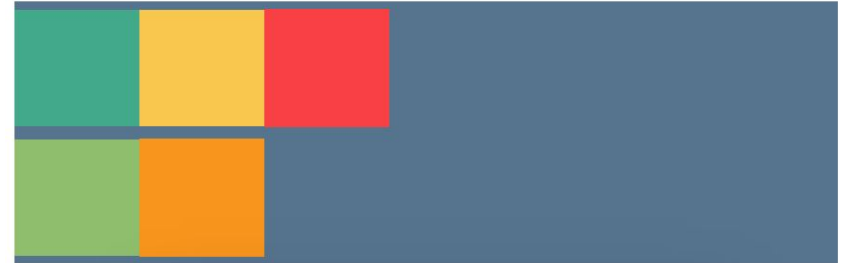
Flex-box: align-content

- Valores
 - flex-start
 - flex-end
 - space-between
 - center
- Obs.: O “flex-wrap: wrap” deve estar setado para que possamos alinhar os elementos entre as linhas ou colunas.

Flex-box: align-content



```
align-content: space-between;
```



```
align-content: flex-start;
```



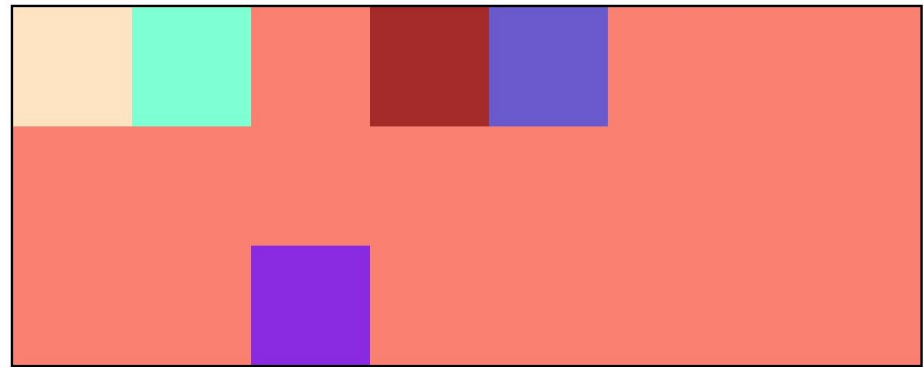
```
align-content: flex-end;
```



```
align-content: center;
```

Flex-box: align-self

- Permite aplicar correções a cada elemento individualmente.
- Valores
 - flex-start
 - flex-end
 - center



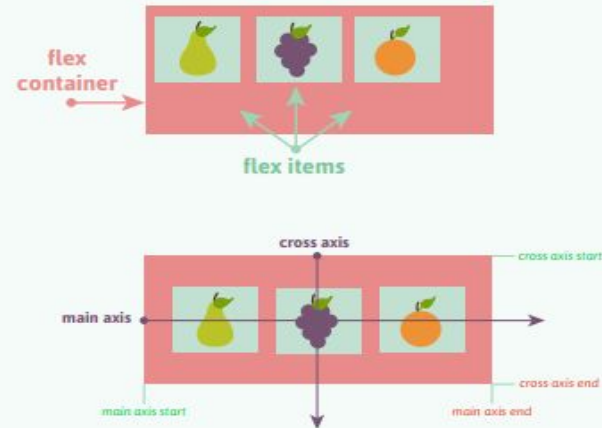
```
#container{  
  background-color: salmon;  
  width: 60%;  
  height: 600px;  
  border: 5px solid black;  
  display: flex;  
  flex-direction: row;  
  margin: 0 auto;  
  flex-wrap: wrap;  
}
```

```
div:nth-of-type(3){  
  align-self: flex-end;  
}
```

Flex-box: cheatsheet

- Flexbox cheatsheet
- Ou Mozilla Docs

Flexbox Cheatsheet Cheatsheet



To activate powers

```
display: flex; on flex container
```

Do you want rows or columns?

rows

```
flex-direction: row; on container
```

OR

```
flex-direction: row-reverse; on container
```



columns

```
flex-direction: column; on container
```

OR

```
flex-direction: column-reverse; on container
```



Flexbox: Resumo de Propriedades

O Flexbox permite organizar elementos de forma dinâmica e eficiente.

1. flex-direction: Define a direção dos itens (eixo principal - colunas ou linhas).

Ex: row (padrão), column, row-reverse, column-reverse.

2. justify-content: Alinha itens no eixo principal.

Ex: flex-start, center, flex-end, space-between, space-around.

3. align-items: Alinha itens no eixo transversal (contrário do principal :-).

Ex: stretch (padrão), center, flex-start, flex-end.

4. flex-wrap: Permite que itens quebrem para a próxima linha.

Ex: wrap, nowrap (padrão).

5. align-self: Aplica alinhamento individual a um item específico.

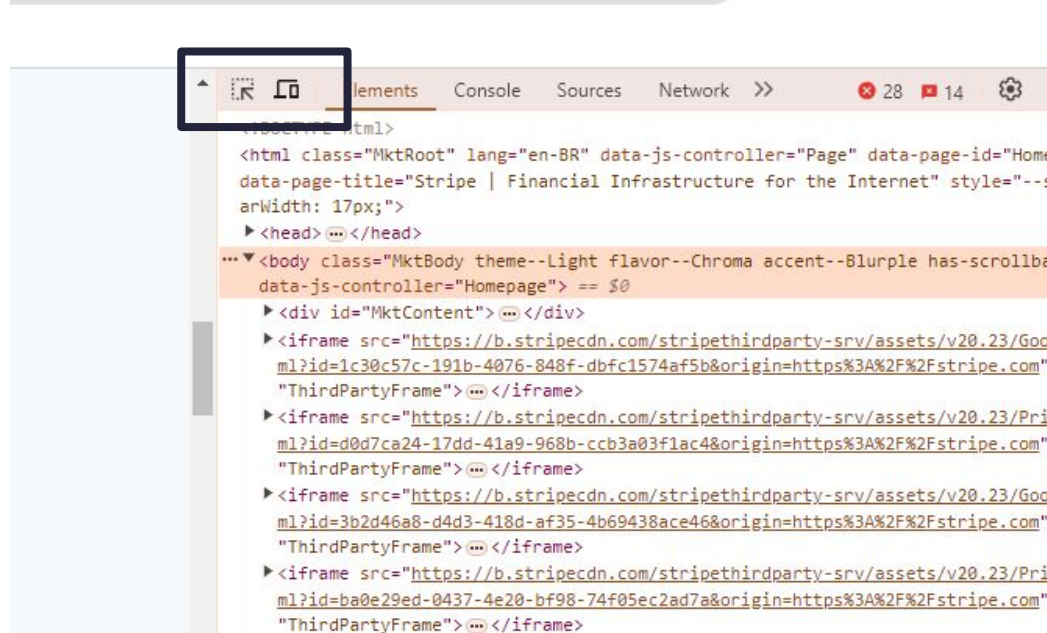
```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
  
.item-especial {  
  align-self: flex-end;  
}
```

Quando usar?

- Centralizar conteúdos vertical e horizontalmente.
- Criar barras de navegação (navbars) com espaçamento igual.
- Layouts responsivos que se adaptam ao tamanho da tela.
- Grids flexíveis de imagens ou cartões (cards).

Design Responsivo: Chrome Debug

- F12 +



- Permite visualizar como seu website se comporta em diferentes tamanhos de tela.